

Lekcja 11 - Spark / PySpark Foundations

2026-09-04

Data Engineering Course

Spark / PySpark Foundations

Kiedy dane są za duże na Polars?
Bronze → Silver transform na PySpark.

■ Problem biznesowy

Masz pipeline lokalny, działa, ale skala rośnie:

```
2 GB -> 500 GB -> 5 TB
```

To nie jest problem jednej funkcji. To jest problem architektury wykonania danych.

W praktyce: da się to zrobić lokalnie, ale nie da się robić to samo w sposób stabilny i powtarzalny przy dużej skali.

■ Dlaczego to jest ważne?

```
Single node = memory + CPU + disk  
Distributed engine = many executors + partitions +  
shuffle
```

Nie chodzi o to, żeby „używać Sparka”, tylko o to, żeby rozumieć, gdzie pojawiają się bottlenecks i kiedy jeden notebook przestaje być wystarczający.

1. Dlaczego Spark
2. Co to jest bronze/silver
3. Lazy evaluation i plan wykonania
4. PySpark DataFrame API
5. Partitioning i output quality
6. Najczęstsze błędy
7. Homework + interview

■ Learning goals

Po tej lekcji umiesz:

```
wyjaśnić kiedy Spark ma sens,  
odróżnić transformation vs action,  
napisać prosty job Bronze -> Silver,  
rozpoznać czy pipeline jest idempotentny,  
uniknąć 4 typowych błędów początkujących.
```

■ First principles

Jedna maszyna ma limit RAM.
Disk i network są dużo wolniejsze niż RAM.

Gdy dane przekraczają możliwości jednej maszyny:

- rośnie spill,
- rośnie time-to-result,
- rośnie ryzyko awarii jobu.

Spark dzieli pracę na wiele executorów i partycji.

■ Polars vs PySpark

```
Polars = single-node, bardzo szybki lokalnie  
PySpark = distributed, skala klastra
```

Reguła praktyczna:

```
start: Polars  
scale: PySpark gdy dane nie mieszczą się w RAM  
jednej maszyny
```

■ Co to jest Spark

Spark to engine, który:

```
czyta dane,  
buduje plan,  
optymalizuje plan,  
wykonuje taski na partycjach,  
zapisuje wynik.
```

Ty opisujesz transformacje. Spark decyduje, jak je wykonać najskuteczniej.

Driver

-> plan i koordynacja

Cluster Manager

-> przydział zasobów

Executors

-> faktyczna praca na danych

W praktyce: **driver** planuje, **executor** wykonuje.

■ Słownik 4 pojęć

```
Job      = uruchomienie po action  
Stage    = część joba między shuffle  
Task     = praca na jednej partycji  
Partition = logiczny fragment danych
```

To są podstawowe słowa do czytania **Spark UI** i **EXPLAIN**.

■ Bronze / Silver / Gold

```
Bronze = raw input, bez "przycinania" biznesowego  
Silver = cleaned + normalized + valid  
Gold   = business-ready aggregates
```

Najpierw robisz poprawny raw import. Potem zbierasz czystość danych. Na końcu liczyć możesz metryki biznesowe.

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("OrdersPipeline")
    .master("local[*]")
    .getOrCreate()
)
```

Praktyka:

- jedna sesja na job,
- `getOrCreate()` zamiast ręcznego `SparkContext`.

■ Ingest Bronze

```
df_bronze = (  
    spark.read  
    .option("inferSchema", "true")  
    .json("data/bronze/orders/")  
)
```

Bronze ma być minimalnie przetworzonym wejściem. Nie wolno mylić raw z "gotowym do raportów".

■ Lazy evaluation

```
df1 = spark.read.json("data/orders.json")  
df2 = df1.filter(df1.status == "completed")  
df3 = df2.select("order_id", "total_amount")
```

Na tym etapie Spark jeszcze nie liczy. To dopiero plan wykonania.

■ Action uruchamia job

```
df3.count()  
df3.show(5)  
df3.write.parquet("output/")
```

Dopiero **action** uruchamia obliczenia. To bardzo ważne dla debugowania i performance.

■ Narrow vs wide

Narrow: filter/select/withColumn

-> zwykle bez shuffle

Wide: groupBy/join/distinct

-> zwykle shuffle

Wide operacje są droższe i szybciej widzą się w **EXPLAIN**.

```
from pyspark.sql import functions as F

(df_bronze
 .withColumn("status", F.trim(F.lower(F.col("
status"))))
 .withColumn("total_amount", F.col("
total_amount").cast("double"))
 .filter(F.col("order_id").isNotNull()))
```

Widzisz tu dokładnie to, czego oczekujemy po **Silver**:
standardyzacja i walidacja wejścia.

■ Partitioning

```
df_silver.write \  
  .mode("overwrite") \  
  .partitionBy("status") \  
  .parquet("output/silver/orders/")
```

Cel:

- mniejsze pliki do odczytu,
- faster pruning w query,
- lepsza organizacja danych wyjściowych.

■ Typowe pułapki

1. collect() na dużym DataFrame
2. pętla Python po recordach
3. brak normalization statusu
4. zapis bez kontroli jakości wejścia

W Spark najczęściej winą jest nie **kod**, tylko **logika danych** i **niezrozumiały plan wykonania**.

Homework i interview

Homework: napisz job Bronze -> Silver + 3 testy
Interview: pokaż, kiedy Spark ma sens, a kiedy nie

Najważniejsze pytanie:

Czy to problem RAM, skali czy bezpieczeństwa danych?

